進階程式設計
Advanced Program Design

L8-2

Chia-Chi Chang
ccchang@cc.ncue.edu.tw

利用後置表示法求值-演算法

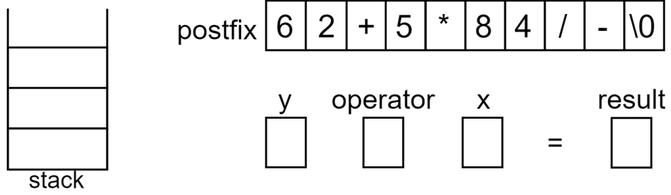
- 先將'\0'插入至後置運算式，並依序儲存至字元陣列postfix
- 堆疊區用來存放計算結果

==========

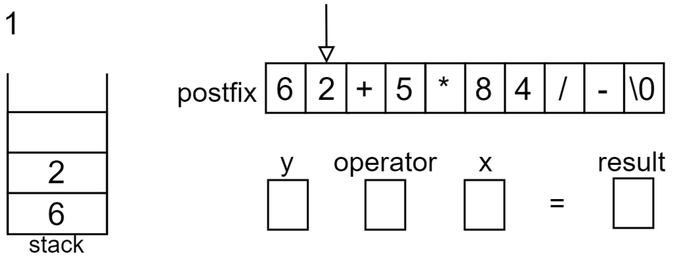
1) 若無遇到'\0'，依序由左至右讀取postfix的後置運算子
    1) 若字元為數字，將其加入至堆疊區。
    2) 若字元為運算子operator，則
        i. 從堆疊區取出2個數字，分別放入x和y
        ii. 計算y operator x後，將計算結果加入至堆疊區
2) 若遇到'\0'，將堆疊區的數值取出，計算結束。

(6+2) * 5 - 8 / 4

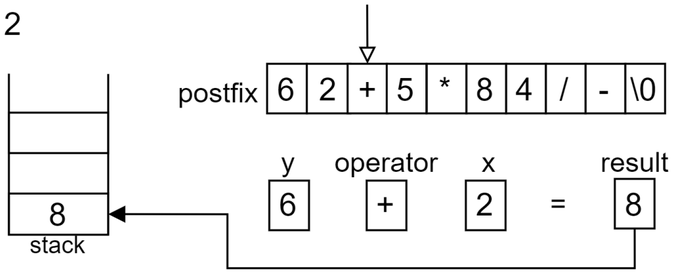
STEP 0 6 2 + 5 * 8 4 / -



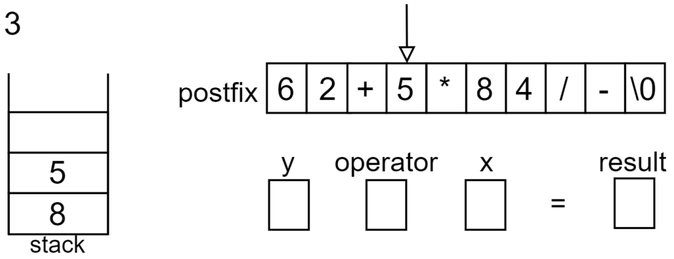
STEP 1



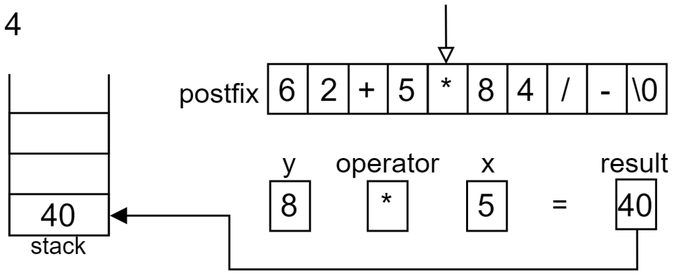
STEP 2



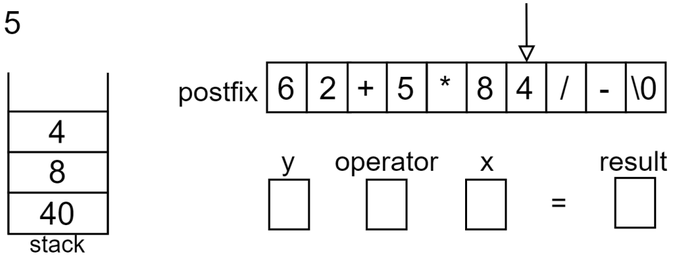
STEP 3



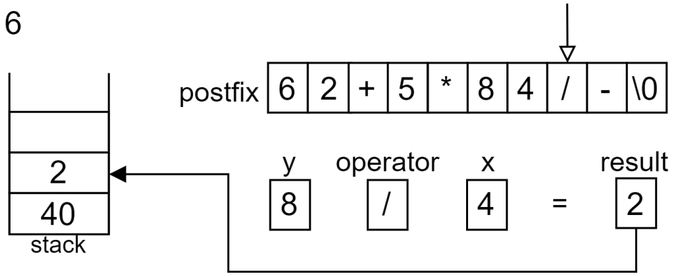
STEP 4



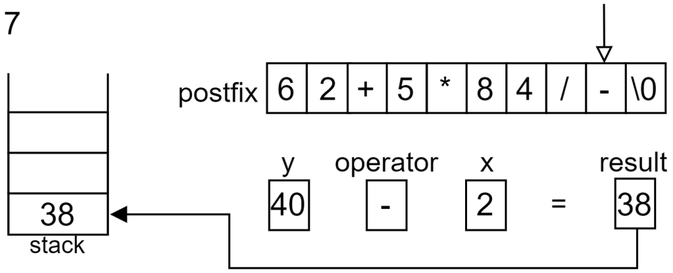
STEP 5



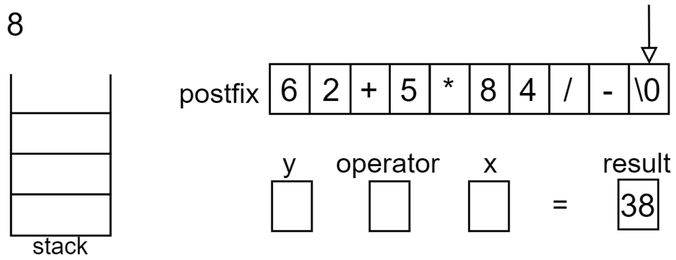
STEP 6



STEP 7

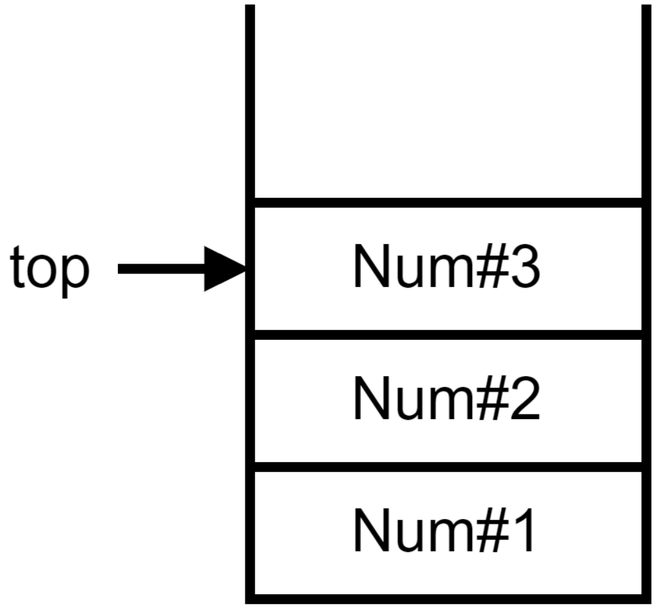


STEP 8

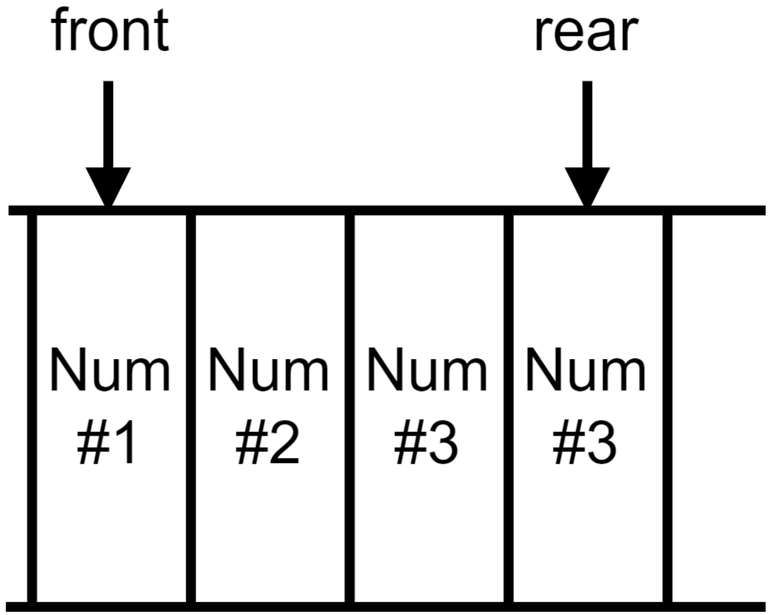


堆疊(Stack)和佇列(Queue)

- 在演算法(Algorithm)中，堆疊(Stack)與佇列(Queue)是常用的資料結構
- 堆疊(Stack)屬於後進先出(Last In Fist Out, LIFO)線性串列，而且只能在頂端(top)進行資料的增加(Push)和刪除(Pop)。
- 佇列(Queue)屬於先進先出(First In First Out, FIFO)線性串列，新增資料發生在後端(rear)，而刪除資料在前端(front)。

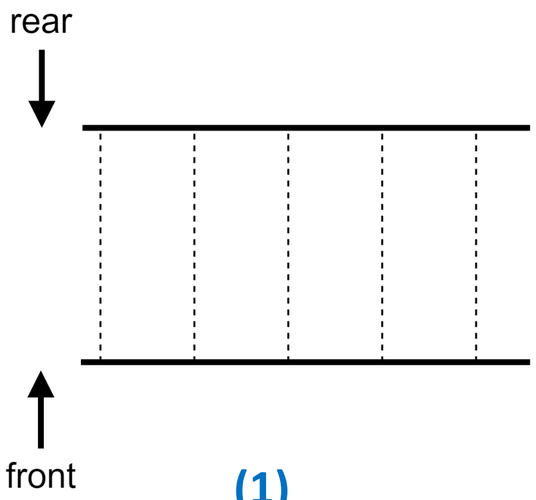


堆疊(Stack)

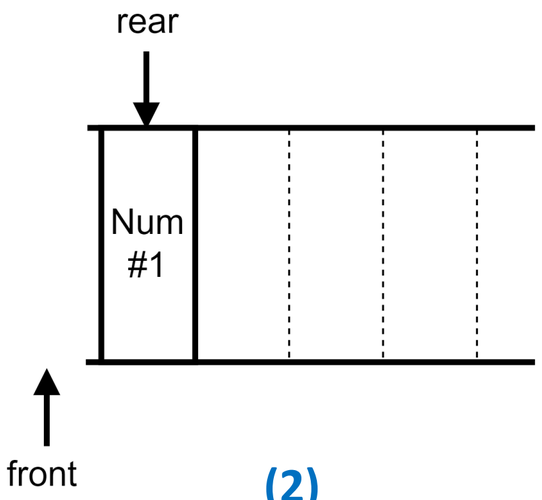


佇列(Queue)

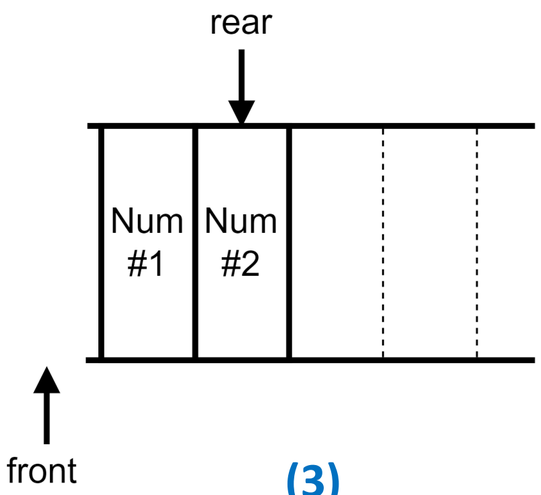
線性佇列(Queue)



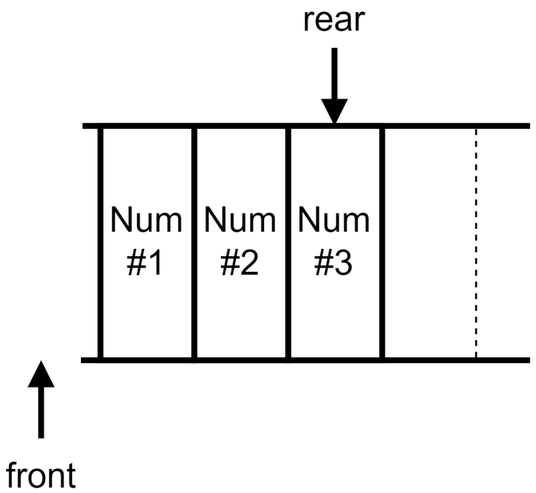
(1)



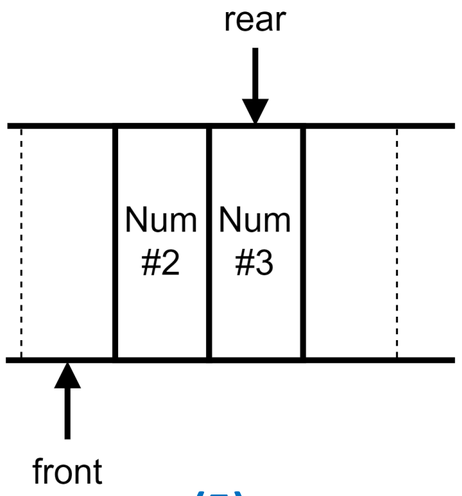
(2)



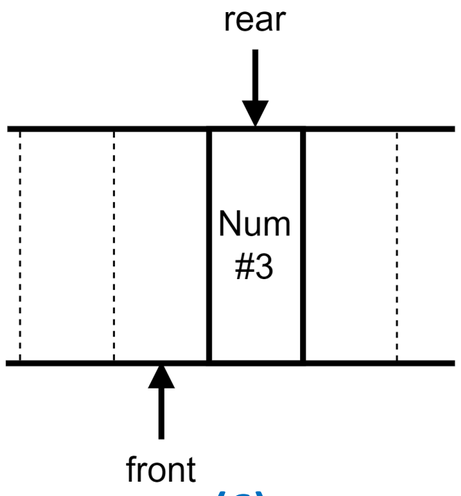
(3)



(4)



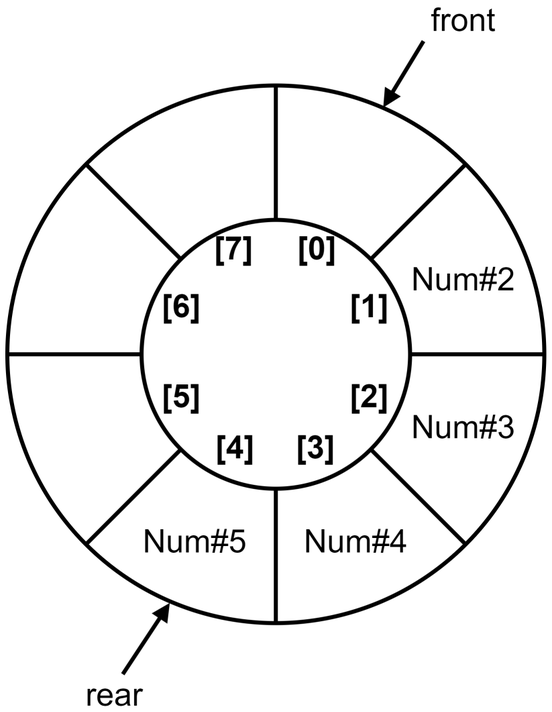
(5)



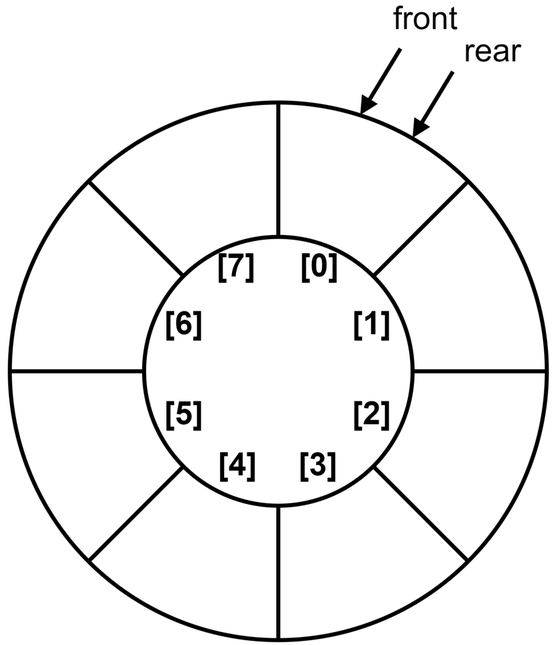
(6)

環狀佇列(Circle Queue)

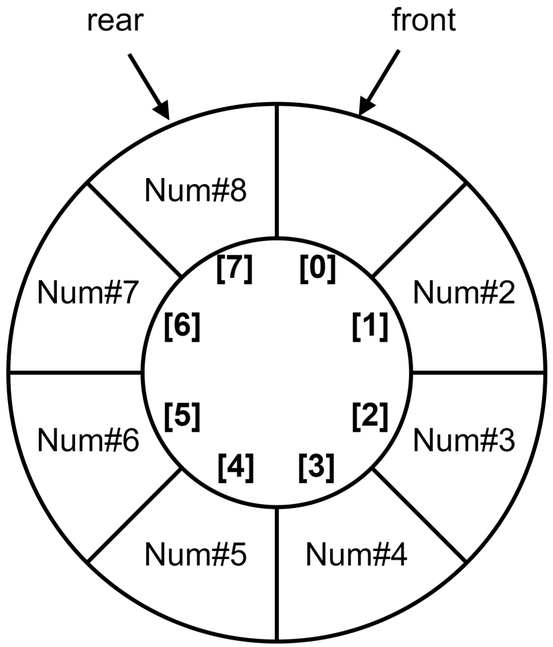
- 線性佇列會遇到佇列已滿(rear已指到尾端)，卻沒有考慮佇列前面仍有空間的不合理狀況。
- 環狀佇列可以改善線性佇列的空間使用率。



環狀佇列(Circle Queue)



環狀佇列已空



環狀佇列已滿

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#define QUEUE_LENGTH    100
typedef struct {
    int16_t front;
    int16_t rear;
    int16_t arr[QUEUE_LENGTH];
} str_cqueue;
```

結構體實現佇列陣列、front、rear

```c
int16_t isEmpty(str_cqueue* queue) {
    return (queue->rear == queue->front) ? 1 : 0;
}
int16_t isFull(str_cqueue* queue) {
    if(queue->front == -1) {
        return ((queue->rear+1) == QUEUE_LENGTH - 1) ? 1 : 0;
    } else {
        return ((queue->rear+1) % QUEUE_LENGTH == queue->front) ? 1 : 0;
    }
}
int16_t add_cqueue(str_cqueue* queue, int16_t num) {
    if(isFull(queue)) {
        return 0;
    } else {
        queue->arr[++queue->rear % QUEUE_LENGTH] = num;
        return 1;
    }
}
int16_t del_cqueue(str_cqueue* queue, int16_t* num) {
    if(isEmpty(queue)) {
        return 0;
    } else {
        *num = queue->arr[++(queue->front) % QUEUE_LENGTH];
        return 1;
    }
}
```

isEmpty, isFull, add_cqueue和del_cqueue函式

```c
int main(int argc, char *argv[]) {
    int16_t loop, m, cmd, num;
    str_cqueue cqueue = {-1, -1,{0}};

    printf("Please enter the command(1:add,2:delete):");
    while(scanf("%hd", &cmd) != EOF) {
        switch(cmd) {
        case 1:
            printf("Enter an integer:");
            scanf("%hd", &m);
            if(add_cqueue(&cqueue,m)) {
                printf("The integer %hd is added into the cqueue!\n",m);
            } else {
                printf("The cqueue is full!\n");
            }
            break;
        case 2:
            if(del_cqueue(&cqueue, &num)) {
                printf("The value %hd has been deleted.\n",num);
            } else {
                printf("The cqueue is empty!\n");
            }
            break;
        default:
            printf("Your command is error!\n");
            break;
        }
        printf("Please enter the command(1:add,2:delete):");
    }
    return 0;
}
```

```
Please enter the command(1:add,2:delete):1
Enter an integer:10
The integer 10 is added into the cqueue!
Please enter the command(1:add,2:delete):1
Enter an integer:77
The integer 77 is added into the cqueue!
Please enter the command(1:add,2:delete):2
The value 10 has been deleted.
Please enter the command(1:add,2:delete):2
The value 77 has been deleted.
Please enter the command(1:add,2:delete):2
The cqueue is empty!
Please enter the command(1:add,2:delete):^Z

--------------------------------
Process exited after 17.31 seconds with return value 0
Press any key to continue . . .
```

The End